AI INFRA RESEARCH SERIES

Jev-Mem

把高频记忆决策与最终语言推理解耦

分析对象

Jev-Mem: System-One-Controlled Agentic Memory for Efficient AI Agents

作者与机构

Dongming Jiang、Yi Li、Bingzhe Li；论文署名团队来自 University of Texas at Dallas。

版本范围

arXiv:2609.23986v1；源码审视固定于 81574eb2（2026-09-22），避免将后续仓库变化冒充论文原始实验。

核心判断

Jev-Mem 研究的是 Agent 的外部长期记忆，不是 Transformer KV Cache。它把记忆组织、路由、预算与停止判断交给结构化控制器，让回答模型主要负责证据综合。最大的研究问题是：节省决策成本的同时，怎样控制错误检索与过早停止。

阅读定位：机制与公式、系统实现、实验口径、局限与复现。

说明：性能数据均标明来源；本报告没有独立训练模型或复跑 GPU 性能。推导与教学算例不作为作者实测结果。

整理日期：2026 年 10 月 31 日

阅读导航

1 它优化的是哪种 memory 2

2 为什么要把控制决策从生成式推理中拆出来 2

3 共享记忆：节点类型与边类型要分开 2

4 类型化控制：Noul 与 Choice 3

5 写入路径：原始证据保留与关系判断 3

6 检索路径：路由、预算、扩展、检查 4

7 停止判断：省成本与漏证据之间的矛盾 4

8 实验数字：分别和谁比较 5

9 公开代码里必须检查的评测风险 6

10 更公平的复现方案 6

11 对 AI Infra 的真正启示 7

12 原始资料 7

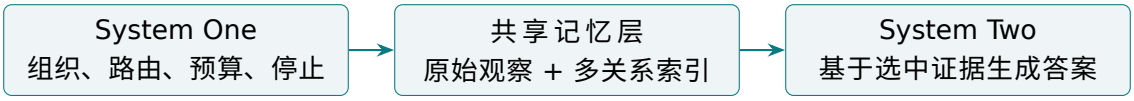
文中的 [S1]、[S2] 等编号指向文末原始资料。引用附带章节、表号或源码位置，便于核对；不同报告的编号独立。

1 它优化的是哪种 memory

Agent 的 memory 可以指很多东西：聊天记录、事实与偏好、工具结果、模型内部 state，或者 GPU 上的 KV。Jev-Mem 面向的是前面这一类**可持久化、可检索的外部观察记录**。它不修改 attention kernel，也没有直接提出 KV 量化或显存分页算法。[S1，§1–3；S2]

一条长期记忆流水线通常是“写入历史 → 建索引 → 检索 → 组织上下文 → 回答”。困难在于中间有很多语义决策：记录属于什么类型？它与旧记录是什么关系？问题需要沿哪种关系寻找？证据是否已经充分？继续找还有没有价值？

这些判断需要理解语言，却不一定需要生成一段自然语言解释。Jev-Mem 将其归到 System One，最终答案生成归到 System Two。这是职责划分，不是把两个名字理解成必须对应某两个固定大小的神经网络。[S1，§3]



最准确的定位：一个带类型化控制器的 Agent 记忆系统，而不是新的 Transformer 或新的 KV Cache 管理器。

2 为什么要把控制决策从生成式推理中拆出来

假设检索有 r 轮，每轮让大模型生成搜索计划、检查结果、再生成下一轮行动。即使最终只回答一句话，中间仍可能发生多次较长的推理和文本解析。

Jev-Mem 的思路是把这些高频问题变成固定格式的决策输出，并将同一状态上的多个问题批量提交。比如对十个候选，输出相关性、新信息量和支持程度，而不是十段自由文本评价。[S2，How it works；S4]

从系统角度，可以写出解释性延迟分解：

$$T_{\text{query}} = T_{\text{route}} + T_{\text{anchor}} + \sum_{d=1}^{r} (T_{\text{assess},d} + T_{\text{expand},d}) + T_{\text{answer}}.$$

优化可以来自每次判断更便宜、轮数更少、证据更紧凑，也可能来自不同的缓存命中。仅看到端到端变快，不能知道是哪一项贡献最大。

控制器不是零成本：它也有推理、网络、序列化、超时和重试；一次返回的字段很多，并不说明计算复杂度不随输入状态变大。部署时要把控制 API 的费用与回答 API 的费用一起计入。

3 共享记忆：节点类型与边类型要分开

3.1 四种关系视角，不是四份重复历史

共享节点可以通过语义、时间、因果和实体关系连接。它们是同一批观察上的不同图视角，而不是将同一句话无条件存进四个独立数据库。[S1，§3.1；S2]

例如两条观察：“旧自行车坏了”和“因为旧车坏了，我买了一辆新车”。它们可能语义相近、涉及同一对象，并存在文本明确表达的因果联系。图中的“因果”是对证据叙述的组织，不是通过干预识别真实因果效应。

时间边也需要谨慎：记录时间、事件时间、文中“昨天／上周”等相对日期可能不同。公开实现会在时间问题的状态中区分 observation time 与事件日期，避免直接把记录日期当答案。[S3，node_state]

3.2 记忆类型可以重叠

每个节点还可以具有 episodic、semantic、procedural、preference 四类分数，即经历、事实知识、过程知识与偏好。一条“今天尝试了工具 A，以后更喜欢用它”既是经历，也表达偏好，不适合强制归成单一类别。[S3]

这种拆分的意义是让查询路线与内容标签各司其职。对“为什么”可以重视因果边，对“以后如何操作”可以重视过程知识，但它们最终可能访问同一条原始记录。

4 类型化控制：Noul 与 Choice

项目使用 Jev 的结构化决策接口。Noul 用于二元命题的分数，Choice 用于互斥选项；普通代码检查输出类型并执行预算与图操作。[S2；S7]

“这个问题需要时间关系吗”和“需要实体关系吗”可以同时为高分，不能把它们错误地归一化为互斥概率。相反，“本次执行合并还是保留”属于有限行动选择。

必须区分三个层次：**格式有效、语义正确、概率已校准**。返回一个合法的 0.95，只保证字段满足接口约定；不自动意味着现实中有 95% 的答案正确率。将这种分数作为停止门槛，需要单独评估校准与错误代价。

控制器的内部参数规模和训练方式，也不应仅由“System One”推断。开源的是记忆系统与调用逻辑；可调用一个模型服务，不等于拿到了完全可自托管的模型权重。

5 写入路径：原始证据保留与关系判断

当前固定快照的默认配置关闭 admission filtering，即 `admission_enabled=false`。系统默认保留有效的原始观察，再选择性建立关系。这避免在写入时把“暂时看起来不重要”的细节不可逆地丢弃。[S2；S5]

写入可概括为：

$$\text{原始观察} \rightarrow \text{类型分数} \rightarrow \text{候选旧节点} \rightarrow \text{关系判断} \rightarrow \text{图更新}.$$

公开配置的候选上限为 10，推断关系阈值为 0.6。确定性的信息，如精确实体标识重合、可直接比较的时间顺序，可以由普通代码处理，不必全部调用语义模型。[S3；S5]

5.1 有界模型候选不代表整个算法有界

`find_candidates` 先用向量索引得到部分结果，但还遍历已有图节点，结合实体、关键词和时间特征，然后排序截取候选。[S3]

因此，模型只判断十个候选，不等于 CPU 只查看十个节点。若已有 N 条记录，候选发现仍包含随 N 增长的工作；完整排序可能达到 $O(N \log N)$。连续写入许多记录时，这会成为新的构建瓶颈。

这不是否定架构，而是指出一个可优化的实现层：实体倒排索引、时间索引、局部候选合并，可能比继续降低模型调用数更有效。性能优化应先 profile 实际时间，不能只数 API 次数。

6 检索路径：路由、预算、扩展、检查

6.1 从多个入口开始，而不是盲目遍历全图

查询先通过向量与关键词得到 anchors，再由控制器预测各关系的需求程度，并分配图遍历预算。固定快照中 anchor 数默认 30，最终普通问题返回 40 条，多跳问题返回 50 条。[S4–S6]

预算含义必须说清：图候选扩展数、返回证据数、控制器调用数、最大耗时是不同约束，不是一个“top-k”统一控制全部成本。

6.2 整数预算如何分配

令关系需求分数为 p_g，激活关系集合为 A，总扩展预算 B。可以将实现概括为先保留最低额度，再按

$$w_g = \frac{p_g^{\gamma}}{\sum_{j \in A} p_j^{\gamma}}$$

分配剩余预算，并用最大余数法处理整数。实际代码还处理激活阈值、零分数、预算过小和关系消融，保证不会超额分配。[S3，allocate_graph_budgets]

教学例子：四种需求为 $(0.3, 0.9, 0.8, 0.5)$，B=80、每个激活关系最低 1、$\gamma = 1$，可分为 $(10, 29, 25, 16)$。问题更需要时间和因果线索，就把较多扩展机会分给对应视角。这个例子不是论文观测数据。

6.3 候选评分关注“补充了什么”

实现结合向量相似度、query 相关性、关系适配、新信息量，以及已有证据支持程度来排序。关键不是只选“最像问题”的句子，而是选择能补上当前证据缺口的句子。[S4]

如果已经取到多条“买了新车”的记忆，再取一条“旧车损坏”可能更有价值。新信息项使评分依赖当前证据集合，区别于固定向量 top-k。

但这些分数并不具有天然可比性。加权平均要求各分数的尺度与误差分布大致合理；阈值和权重需要在独立验证集检查，而不能在测试集上反复调到最好。

7 停止判断：省成本与漏证据之间的矛盾

控制器同时预测证据充分 s、继续价值 u、缺失程度 m、矛盾程度 c。固定实现中的充分性条件为

$$s \geq 0.95 \quad \wedge \quad m < 0.15 \quad \wedge \quad c < 0.15.$$

还可以因继续价值不足，或节点、边、深度、调用次数、延迟达到上限而停止。[S4–S5]

停止原因	意味着什么	不意味着什么
证据充分	控制器认为足够回答	真实答案必然正确
继续检索无益	当前路线继续找的收益低	已经知道所问事实
预算／时间耗尽	达到资源保护边界	应当猜一个答案
无更多候选	当前检索空间耗尽	事实在全部历史中不存在

一个系统级改进是让停止原因影响最终回答策略：充分时正常回答；缺证据或预算耗尽时明确不确定性，而不是只在日志中记录停止类型。

7.1 为什么强回答模型无法修复所有控制错误

如果控制器把真正相关的节点过滤掉，System Two 从未看到这份证据，再强也可能无法还原事实。整个系统的质量上限受 evidence recall 约束。

因此，控制错误的代价不相同：多取一条无关记录可能只增加 token；过早停止可能永久切断关键证据。应分别测 “无关扩展率”“漏证据率”“错误充分性率”，而不是只看控制器平均分类准确率。

8 实验数字：分别和谁比较

下表是论文 Tables 1–2、固定 README 一致报告的 LoCoMo 结果，回答模型为 GPT-4o-mini。质量指标是 LLM-as-a-Judge 分数，查询时间包括检索与答案生成。[S1–S2]

方法	Overall 分数	构建秒数	平均查询秒数
Full Context	0.481	不适用	1.74
A-MEM	0.580	3,636	2.26
MemoryOS	0.553	3,276	32.68
Nemori	0.590	1,044	2.59
MAGMA	0.700	1,404	1.47
Jev-Mem	0.777	158	0.93

11.0% 是 $(0.777 - 0.700)/0.700$，即对 MAGMA 的相对提升，绝对差为 0.077。6.6× 是构建时间 $1044/158$，对比的是最快构建基线 Nemori。36.7% 是查询延迟相对 MAGMA 的下降；在线速度比约 1.58×，不是 6.6×。

类别成绩也不是全面领先：时间类 Jev-Mem 为 0.637，低于 MAGMA 的 0.650；其他四类领先。Adversarial 类的高分代表这个数据集中的相应问答能力，不证明已经解决提示注入或生产安全。[S2，分类表]

这些分数不能与其他论文使用不同 judge、不同回答模型的 LoCoMo 数值直接拼成排行榜。总体分数也不应不看样本权重就由五个类别做算术平均。

9 公开代码里必须检查的评测风险

9.1 Best-of-N 的参考答案辅助选择

README 明确提醒：继承的默认 best-of-N=3 可以使用参考答案辅助挑选候选，单答案评测应设 `--best-of-n 1`。`test_harness.py` 存在多次生成后用 reference 做 LLM judge 或 F1 选择的路径。[S2；S6]

能确认的是代码中存在这条路径；**不能因此直接断言论文的 0.777 一定来自该路径**。要判断实际实验，仍需要对应运行命令、配置与日志。

复现时应保证 reference 只参与事后评分，不能进入检索、生成或候选选择。否则得到的不是线上可用的单次问答能力。

9.2 证据预算不一致会影响归因

固定测试代码中，Jev 读取路径默认返回 40/50 条，另一条基线路径为 15/30 条。更多证据可能改善召回，也会增加上下文与生成开销。[S6]

因此应同时做等 token 预算和等检索资源预算对照。只固定“条数”也不够，因为每条长度不同。没有这些对照，不能把全部质量差异只归因于新控制器。

9.3 运行示例不等于完整论文复现

离线 demo 使用 mock 决策和 embeddings，只验证流水线可运行。真实查询依赖 Jev 服务与回答模型；`jev-latest` 这种别名会变化，必须记录服务版本和时间。README 的十问示例还排除了 adversarial 类，不能当作完整五类基准。[S2]

LongMemEval runner 的存在也不等于论文已经报告了相应结果；文档说明它使用的评分不是官方 LongMemEval 口径。报告 v1 的正文与表格还存在个别类别数值差异，本文采用表格与固定 README 一致的值，不替作者拼接更优结果。

10 更公平的复现方案

先固定一个记忆图，比较规则控制器、Jev 控制器与固定 top-k；这样能隔离读取策略，不让构图差异混进来。再固定读取策略，比较不同写入关系策略。最后比较整体系统。

实验	固定项	主要观察
关闭自适应停止	相同 anchors、评分、上下文预算	少搜多少，漏证据多少
等 token 预算	回答模型、prompt、采样次数	收益是否来自多塞证据

实验	固定项	主要观察
冷／热缓存	同一问题集、服务版本	缓存是否主导延迟
记忆规模扫描	相同单条长度与关系密度	全图扫描何时成为瓶颈
故障与超时	固定 fallback 策略	质量、p95/p99 与费用变化

数据隐私也属于系统验收：真实运行会把文本发送到配置的控制与回答服务。组织内部记录、工具日志和密钥不能因为“只是构图”就被当成无敏感信息。

11 对 AI Infra 的真正启示

这篇的价值是把成本优化前移到“该不该执行这次推理或检索”。它与 GEMM 加速并不冲突：前者减少无效工作，后者加速必须完成的工作。

可以将目标写为本报告的分析形式：

$$\min(\text{控制成本} + \text{检索成本} + \text{回答成本}), \quad \text{s.t. evidence recall、答案质量与尾延迟达到要求.}$$

有价值的研究不必重新造一个 Agent：消除候选发现的全图扫描、按错误代价校准停止器、实现可审计的控制 trace，都是具体系统问题。但它们需要新实验，不应被包装成本文已经验证的贡献。

12 原始资料

[S1] 论文 v1。 arXiv 2609.23986v1，重点 §3、Tables 1–2 与局限说明。

[S2] 固定版本 README。 项目说明与评测警告。

[S3] 写入与预算策略。 memory/jev_mem_policies.py。

[S4] 检索控制器。 memory/jev_mem_retrieval.py。

[S5] 默认配置。 config/jev_mem.json。

[S6] 评测执行。 memory/test_harness.py。

[S7] 类型化决策接口。 TypeSafe 官方文档。

本报告没有独立运行 LoCoMo 或服务调用。对代码风险的讨论不等于断言作者实验使用了有问题的配置；实际归因需要原始运行记录。